

A Voyage to the Kernel



Part 16

Segment: 3.5, Day 15

We are dedicating today's voyage to learning more theory and clarifying the doubts of beginners.

Inter-Process Communication (IPC)

The Linux Inter-Process Communication (IPC) mechanism is an essential tool that allows the 'sync operation' between processes and facilitates sharing of resources. IPC is also employed for exchanging information with another program. Thus, this is implemented using shared resources, kernel data structures and wait queues. Refer to Figure 1 for the subsystem structure of IPC.

Here are the main IPC implementation methods (note: the following are generalised descriptions):

- **Signals:** These are the oldest form of UNIX IPC and they are actually asynchronous messages sent to a process.
- **Wait queues:** The system can use this to put a process in sleep mode, if the corresponding operation is not yet completed (for example, bottom-half handling by the process scheduler).
- **Pipes (and named pipes):** By using a pipe connection, a connection-oriented bi-directional data transfer between any two processes (or, via a named pipe in the filesystem) can be done.
- **File locks:** This IPC allows a process to declare a file (or part) as read-only to all other processes. Hence the one that holds the lock can only modify it.
- **Unix Domain sockets:** This also is a connection-oriented data-transfer mechanism like the pipe, and the implementation is akin to the INET sockets.
- **System V IPC:**
 - **Semaphores:** This IPC model allows the creation of arrays of semaphores.
 - **Message queues:** This is a connectionless data-transfer model. A message is essentially a sequence of bytes. And by reading the message queues, it is retrieved. This also uses an associated type that restricts the message read.

- **Shared memory:** As the name suggests, this mechanism facilitates accessing a given portion of physical memory by several processes.

The following code shows the notification mechanism for IPC namespaces:

```
#include <linux/msg.h>
#include <linux/rtnetlink.h>
#include <linux/notify.h>
#include <linux/nsproxy.h>
#include <linux/ipc_namespace.h>

#include "util.h"

static BLOCKING_NOTIFIER_HEAD(ipcn_chain);

static int ipcn_callback(struct notifier_block *self,
                        unsigned long action, void *arg)
{
    struct ipc_namespace *ns;

    switch (action) {
    case IPCNS_MEMCHANGED: /* amount of lowmem has changed */
    case IPCNS_CREATED:
    case IPCNS_REMOVED:
        /*
         * It's time to recompute msgmn
         */
        ns = container_of(self, struct ipc_namespace, ipcn_nb);
        /*
         * No need to get a reference on the ns: the 1st job of
         * free_ipc_ns() is to unregister the callback routine.
         * blocking_notifier_chain_unregister takes the wr lock to do
         * it.
         * When this callback routine is called the nl lock is held by
         * blocking_notifier_call_chain.
         * So the ipc ns cannot be freed while we are here.
         */
    }
```